FLOWS - Flood Likelihood from pre-compiled Watershed Structure - Documentation

Thiago Ricardo Borges

September 6, 2026

Contents

1 Introduction

Introduction text

2 Building

Now we begin the actual process of making the model.

2.1 First thoughts

2.1.1 The challenge of flood prediction

Flood prediction is one of the most computationally demanding problems in environmental modeling. Modern hydraulic models must simultaneously take in consideration terrain, rainfall, infiltration, surface runoff, river networks, soil properties, and the interaction between all these components through out time. Many systems rely on models that solve large systems of equations over millions of spatial cells. Depending on the resolution and simulation domain, these models may require powerful workstations, clusters and/or expensive computing infrastructure.

Examples include:

- The HEC-RAS hydraulic model, widely used for river and floodplain simulations.
- WRF-Hydro, which couples weather and hydraulic processes.
- National and regional flood forecasting systems operated by governments and research institutions

While these models achieve high levels of accuracy, they usually require significant resources, specialized knowledge and extensive input datasets

2.1.2 Why Flood Estimation for low-end hardware matters

Many communities, schools, researchers, local governments and independent developers do not have access to high-performance computers. In emergency situations, decision-makers may benefit from a model that sacrifices some physical complexity in exchange for:

- Faster execution
- Lower hardware requirements
- Easier deployment
- Greater accessibility
- Large-scale applicability

The goal is not necessarily to replace advanced hydrodynamic simulation, but to provide a practical first-order estimate of flood-risk

2.1.3 The Core idea

Water naturally accumulates in topographic depressions. Instead of continuously solving fluid dynamics equations across an entire terrain, this project explores the possibility of pre-processing terrain into a network of natural storage basins. Each basin can be described by:

- Area
- Storage capacity
- Elevation - Volume relationship
- Overflow evaluation

Once this network is compiled, rainfall forecats can be converted into water volumes and handled by our already calculated basin properties. This shifts much of the computation cost from runtime to a one-time preprocessing stage

2.1.4 Proposed Approach

- Obtain a Digital Elevation Model (DEM) with an acceptable resolution
- Identify topographic bowl-like depressions (here, we will just call them "basins")
- Calculate storage - volume relationships for each basin
- Build a graph representing basin connectivity
- Incorporate rainfall forecasts and infiltration estimates
- Estimate water accumulation and flood risk
- Test on real-life events.

2.1.5 Long-term goal

The objective is to investigate whether a graph-based representation of terrain storage can provide useful flood-risk estimates while remaining lightweight enough to run on consumer hardware. Such an approach could make flood-awareness tools more accessible to small municipalities, educational institutions, researchers and citizen initiatives.

2.2 Initial steps / handling geoTIF files

First of all, we got to check out the first item in out list: obtaining a DEM with acceptable resolution. Now this is where things can get tricky: some countries have decent, high-quality DEM's made with LiDAR and other fancy methods that will provide us with extremely accurate information. But for the big majority (including

me), those same resources are not easily accessible (if they can at all be found). In those cases, you would probably be best of with "open for public" elevation models such as Copernicus GLO-30 (that is very good, considering its freely available), but take in consideration that it may suffer from vegetation bias, and other inaccuracies that come with initiatives like this. Now, in my (the author) scenario, we are kind of lucky, since we have ANADEM.

ANADEM is a DEM brought to us by the Brazilian National Waters and Basic Sanitation Agency (ANA) in collaboration with the Federal University of Rio Grande do Sul (UFRGS). It is a refined version of Copernicus GLO-30, with reduced vegetation bias and developed with hydrological analysis in mind, available for South America.

Before continuing, it's important to remind the reader to be aware of DEM origins and quality, since those factors can and will affect the outcome of a model. It's also crucial to mention that, even though the goal of this documentation is to provide instructions and notes of FLOWS's construction to allow replication, the reader should have in mind that everything we use here was specifically made for Brazilian territory, so before adapting this project to your own needs please use parameters and resources for your own specific region/country.

Also let's have in mind that, for this project, hardware specs are so far:

- Lenovo Thinkpad E470
- Intel Core i5-7200U
- 8GB of DDR4 RAM
- 120GB SATA Solid State Drive
- Fedora Workstation 44

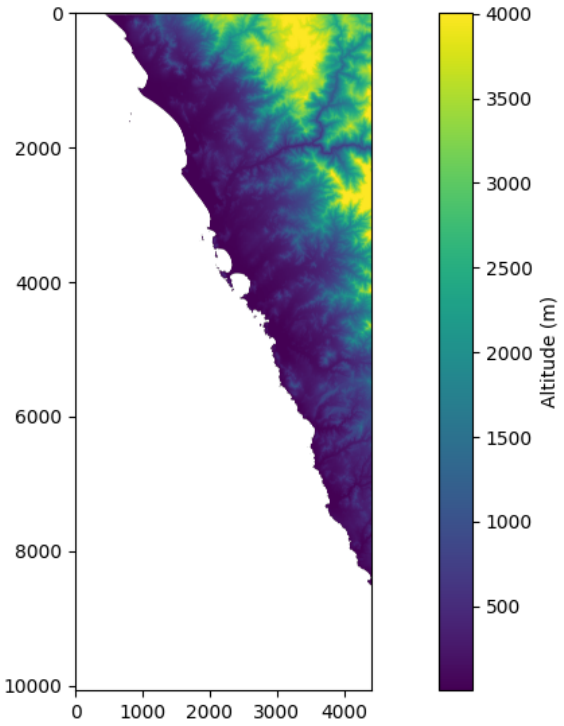
We need, as this project advances, to have this specs in mind.

Now, let's get to the really interesting stuff. So far, all that we want to do is find all the topographic depressions (let's call them basins), and get from them all the important data, like volume to water height relationship (probably the most important one so far). Later on, we will also analize other factors like water infiltration and model accuracy.

First things first, ANADEM is provided as several .tif tiles that, together, form the whole DEM, as a single big file could easily be 60GB in size or more (We are talking of South America here - I wasn't able to find a DEM focused on hydrological analysis for Brazilian territory alone) . So, to get started, we'll make a simple python script (we'll be working with this language for the rest of the guide since libraries for our specific use are widely available, thus making our whole journey easier) that will read a single tile and output it in a visual format:

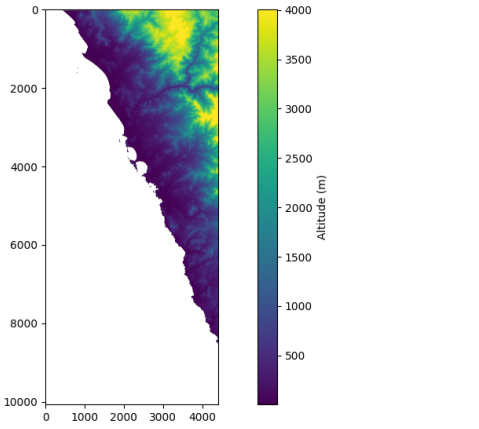
```python
import rasterio
import matplotlib.pyplot as plt
import numpy as np

# This code tests reading a .tif file, and generates a cool
    image with it!

# We are here using a .tif file named mdt.tif.

with rasterio.open("mdt.tif") as src:
print("Bounds:", src.bounds)
print("CRS:", src.crs)
print("NoData:", src.nodata)
dem = src.read(1)

dem = np.where(dem == -9999, np.nan, dem)

valid = dem[~np.isnan(dem)]

print("Min:", np.min(valid))
print("Max:", np.max(valid))
print("Mean:", np.mean(valid))

plt.figure(figsize=(12,8))
plt.imshow(
    dem,
    vmin=np.percentile(valid, 2),
    vmax=np.percentile(valid, 98)
)
plt.colorbar(label="Altitude (m)")
plt.show()

```

This code will generate us an output of this format:

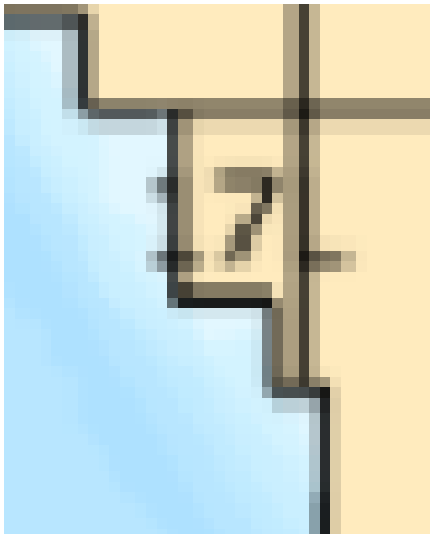


(x, y) = (4.28e+03, 7.05e+03)
[2.6e+02]

Now please take in consideration that this specific tile had some noData zones - areas of the tile with no specific information about them, so they appear in blank. This should not be important for us now, since this tile in specific is located on the west coast of South America, and a visual inspection gives us that this noData zones are actually the ocean. Therefore, we will not worry about them here in this part of the path.



(x, y) = (4.28e+03, 7.05e+03)
[2.6e+02]



Now, we have to think how to actually take all of the info we've downloaded and turn it into an actually usable set of data. Some ideas would be:

- Load everything into a script and just find the basins → Well, not so fast there partner. As we've seen earlier, everything together will render us 60GB of information, all of that loaded at the same time, requiring a lot of storage and RAM. Since we don't have hardware like that available, we'll have to improvise.

- Load each individual tile one at a time → In theory this idea is pretty good! We would just need to load each downloaded tile individually and then find all the basins that are present in it. But there is a pretty important caveat: This solution is a victim to a widely discussed thing in hydrology and topography called Boundary Limitations → what would happen if a basin is cut in half by two adjacent tiles? This would generate inaccuracies that will be costly later. Therefore, not a totally viable solution.

- Load a central chunk of space and expand the borders if needed → How would this work: Basically we start to look for basins in an individual chunk in an individual tile that we will call the core box. We will try and look for all basins inside that area of space, and if any basin touches the boundaries delimitated by the core box, we will expand our search space by adding a buffer to the sides. Then, we will look for basins that still belong to the original core box, but instead check wheter any one touches the new boundaries delimitated by the core box + buffer to the sides. If sucess, we get a file with a know area that does not suffer boundary limitations, but if we fail, we increase the buffer and try again. We repeat this process until we get a sucess.

This last idea seemed pretty good in theory and seems like the most straightfoward option considering our scenario, but there are still some problems to it:

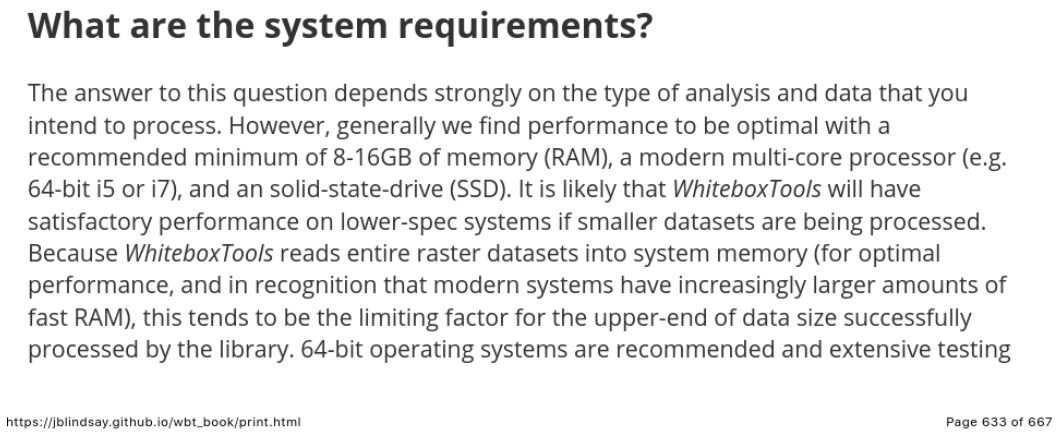
1. Terrain processing → Finding basins may seem straightfoward, but actually applying all the things we learned into code, while having to mess with different file types and concepts that no one without a geography major would ever know (author included) is a completely different history.

2. How do we decide which basins should belong to the initial core box, and how do we deal with basins that belong (in area) to two different core boxes?

3. Way to big loops → What happens if a basin is just so big and we increase the buffer so much that the computer runs out of RAM?

But after some time brainstorming this topic, we are able to come up with a solution to each of these problems:

1. WhiteboxTools → Developed in Rust, WhiteboxTools is an advanced geospatial analysis library designed for hydrological processing. It comes with a range of tools, some made specifically for our objetive of finding bowl-like depressions. Also, it can be used as a Python library! But not everything is perfect, and WhiteboxTools will still try to load everything we put at it on RAM without second thoughts. This will still break our computer if we try to load the entire DEM.



2. Geometry! → Every geometric shape has a unique point called the centroid (in some cases also known as a center of mass). What we can do is check, for each basin with a centroid belonging inside the core box boundaries, wheter it touches the borders of the bigger core box + buffer boundaries. And since the centroid is unique, we can guarantee that each basin will belong in a single core box.

3. A little bit of math (and hope) → if we sum all the tile sizes, we get that our whole DEM, for the entire South America, weights around 63GB, Following that logic, the biggest floodable plain also in South America (Pantanal) would weight around 0.9GB in size if it were its own big basin. So in theory, and big enphasis in theory, we should be able to run this program if we slice the big DEM in smaller individual core boxes the right way and optimze resource usage on our computer. But there is only one way to find out!

But you may still be wondering to yourself: "Okay, but how will we make our code look at all of our downloaded tiles and treat is as one big individual DEM without exceding RAM and storage?" - for that question, we will also introduce the concept of a VRT (Virtual Raster).

A VRT is a lightweight XML-based file created by the GDAL ecosystem that acts essentially as a virtual mosaic. Instead of holding raw pixel data, it stores metadata and instructions on how multiple individual downloaded tiles should be aligned and

organized to appear as a single, continuous, massive image. It basically works as follows (in very simple terms):

- Zero Space Duplication: We don't need to physically merge all tiles into a 60GB monster on disk. The VRT dynamically points to where each original tile is saved.
- On-Demand Reading: When the code starts processing the topography, it reads through the VRT, managing block reading and streaming only the necessary pixels directly from the disk at that exact moment.
- Seamless Global View: To the algorithm, the VRT makes the entire dataset look like a single continuous file, bypassing memory bottlenecks and removing the need to manage tile transitions manually.

So with all of that in mind, let's get started.

2.3 Finding basins

First of all, we have to create our VRT. There are several methods out there, but here it was chosen to use the terminal tool. To install it, you must follow instructions for your specific system. In this guide, we are on Fedora Workstation 44:

```
sudo dnf update && sudo dnf install gdal
```

After that, we must go to the folder containing all of our downloaded .tif files, and run the following command:

```
gdalbuildvrt <output file>.vrt *.tif
```

In this guide, we've chosen to name the output file as simply output.vrt, therfore we ran:

```
gdalbuildvrt output.vrt *.tif
```

```
untitledmacaw@fedora:~/Downloads$ ls
anadem_mgrs.zip    anadem_v1_18N.tif  anadem_v1_20F.tif  anadem_v1_21K.tif  anadem_v1_23J.tif
anadem_tiles.png   anadem_v1_18P.tif  anadem_v1_20G.tif  anadem_v1_21L.tif  anadem_v1_23K.tif
anadem_v1_17L.tif  anadem_v1_19F.tif  anadem_v1_20H.tif  anadem_v1_21M.tif  anadem_v1_23L.tif
anadem_v1_17M.tif  anadem_v1_19G.tif  anadem_v1_20J.tif  anadem_v1_21N.tif  anadem_v1_23M.tif
anadem_v1_17N.tif  anadem_v1_19H.tif  anadem_v1_20K.tif  anadem_v1_21P.tif  anadem_v1_24K.tif
anadem_v1_18F.tif  anadem_v1_19J.tif  anadem_v1_20M.tif  anadem_v1_22H.tif  anadem_v1_24L.tif
anadem_v1_18G.tif  anadem_v1_19K.tif  anadem_v1_20N.tif  anadem_v1_22J.tif  anadem_v1_24M.tif
anadem_v1_18H.tif  anadem_v1_19L.tif  anadem_v1_20P.tif  anadem_v1_22K.tif  anadem_v1_25L.tif
anadem_v1_18K.tif  anadem_v1_19M.tif  anadem_v1_21F.tif  anadem_v1_22L.tif  anadem_v1_25M.tif
anadem_v1_18L.tif  anadem_v1_19N.tif  anadem_v1_21H.tif  anadem_v1_22M.tif  RJ_2016.pdf
anadem_v1_18M.tif  anadem_v1_19P.tif  anadem_v1_21J.tif  anadem_v1_22N.tif  Sistema-Brasileiro-de-Classificacao-de-Solos-2025.pdf
untitledmacaw@fedora:~/Downloads$ gdalbuildvrt output.vrt *.tif
0...10...20...30...40...50...60...70...80...90...100 - done.
untitledmacaw@fedora:~/Downloads$ ls
anadem_mgrs.zip    anadem_v1_18P.tif  anadem_v1_20H.tif  anadem_v1_21N.tif  anadem_v1_24K.tif
anadem_tiles.png   anadem_v1_19F.tif  anadem_v1_20J.tif  anadem_v1_21P.tif  anadem_v1_24L.tif
anadem_v1_17L.tif  anadem_v1_19G.tif  anadem_v1_20K.tif  anadem_v1_22H.tif  anadem_v1_24M.tif
anadem_v1_17M.tif  anadem_v1_19H.tif  anadem_v1_20M.tif  anadem_v1_22J.tif  anadem_v1_25L.tif
anadem_v1_17N.tif  anadem_v1_19J.tif  anadem_v1_20N.tif  anadem_v1_22K.tif  anadem_v1_25M.tif
anadem_v1_18F.tif  anadem_v1_19K.tif  anadem_v1_20P.tif  anadem_v1_22L.tif  output.vrt
anadem_v1_18G.tif  anadem_v1_19L.tif  anadem_v1_21F.tif  anadem_v1_22M.tif  RJ_2016.pdf
anadem_v1_18H.tif  anadem_v1_19M.tif  anadem_v1_21H.tif  anadem_v1_22N.tif  Sistema-Brasileiro-de-Classificacao-de-Solos-2025.pdf
anadem_v1_18K.tif  anadem_v1_19N.tif  anadem_v1_21J.tif  anadem_v1_23J.tif
anadem_v1_18L.tif  anadem_v1_19P.tif  anadem_v1_21K.tif  anadem_v1_23K.tif
anadem_v1_18M.tif  anadem_v1_20F.tif  anadem_v1_21L.tif  anadem_v1_23L.tif
anadem_v1_18N.tif  anadem_v1_20G.tif  anadem_v1_21M.tif  anadem_v1_23M.tif
untitledmacaw@fedora:~/Downloads$
```

The next thing we should do is also install Whitebox:

```
pip install whitebox
```

```
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ pip install whitebox
Collecting whitebox
  Downloading whitebox-2.3.6-py2.py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: Click>=6.0 in ./.venv/lib64/python3.14/site-packages (from whitebox) (8.4.1)
Downloading whitebox-2.3.6-py2.py3-none-any.whl (74 kB)
Installing collected packages: whitebox
Successfully installed whitebox-2.3.6

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ pip freeze > requirements.txt
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$
```

Perfect! So now let's move on to the next step. Our current code workflow can be described as follows:

1. Chose 4 values (min x, min y, max x, max y) that will delimitate our core box
2. Run a loop where we:
   - Increase the buffer by a fixed value
   - Convert the piece from the original VRT delimited by our core box + buffer from referencing system EPSG:4326 (degrees) to referencing system EPSG:5800 (meteres). This is important because calculations later on involving degrees will gives us inaccuracies.
   - Check if the current buffer is way too big (we will set a buffer limit for that)
   - Find all bowl-like depressions + their contribution areas and, for each one, find its centroid
   - Filter results where the centroid belonds to the original core box

- For those results, we check if any one touches the border of the core box + buffer area
- If yes, we run another cycle. If not, we break the loop

3. We save our results and analyse each individual basin.

With that in mind, let's start translating that to code. We are able to initially come uo with this:

```
import os
import geopandas as gpd
from shapely.geometry import box
from osgeo import gdal
from whitebox import WhiteboxTools
import rasterio
import numpy as np
import scipy.ndimage as ndimage
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.colors import ListedColormap

gdal.UseExceptions()

def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
    """
    This will cut the DEM according to topography, preserving basin boundaries (AKA no basins get cut)
    Criteria for cutting: All basins with a centroid that belongs to core tile A will be grouped together on the same TIF

    :param path_vrt: Path to our .vrt file
    :param out_dir: Path for our output
    :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
    :param tile_id: Unique tile ID

    """

    os.makedirs(out_dir, exist_ok=True)
    minx, miny, maxx, maxy = core_bounds_deg

    # Projects Core Tile limist from 4326 (degress) to 5800 (meters)
    df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny, maxx, maxy)]}, crs="EPSG:4326")
```

```
    df_core_5800 = df_core.to_crs("EPSG:5880")
    core_bounds_5800 = df_core_5800.total_bounds # (minx, miny, maxx, maxy) em metros

    # Dynamic buffer initial setup
    buffer_current = 0.1
    buffer_steps = 0.1
    buffer_limit = 4
    sucess = False

    wbt = WhiteboxTools()
    wbt.set_working_dir(os.path.abspath(out_dir))
    wbt.verbose = False

    while not sucess and buffer_current <= buffer_limit:
        print(f"[{tile_id}] Trying to process with a margin of {buffer_current}")

        # Define limits
        buf_minx = minx - buffer_current
        buf_miny = miny - buffer_current
        buf_maxx = maxx + buffer_current
        buf_maxy = maxy + buffer_current

        # Names of temporary files
        filename_dem = f"temp_{tile_id}_dem.tif"
        filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
        filename_d8 = f"temp_{tile_id}_d8.tif"
        filename_sinks = f"temp_{tile_id}_sinks.tif"
        filename_basins = f"basins_final_{tile_id}.tif"
        filename_depth = f"depth_final_{tile_id}.tif"

        path_dem = os.path.join(out_dir, filename_dem)
        path_basins = os.path.join(out_dir, filename_basins)

        try:
            # Cuts OG VRT
            gdal.Warp(
                destNameOrDestDS=path_dem,
                srcDSOrSrcDSTab=path_vrt,
                format='GTiff',
                outputBounds=(buf_minx, buf_miny, buf_maxx, buf_maxy),
                outputBoundsSRS='EPSG:4326',
```

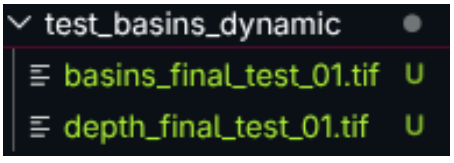
```
                dstSRS='EPSG:5880',
                xRes=30, yRes=30,
                creationOptions=['COMPRESS=DEFLATE', 'TILED=YES']
            )

            # WhiteboxTools pipeline here
            print(f"    -> Running WhiteboxTools ...")
            wbt.breach_single_cell_pits(dem=filename_dem, output=filename_dem_clean)
            wbt.d8_pointer(dem=filename_dem_clean, output=filename_d8)
            wbt.sink(i=filename_dem_clean, output=filename_sinks, zero_background=True)
            wbt.watershed(d8_pntr=filename_d8, pour_pts=filename_sinks, output=filename_basins)

            # Colision Test pipeline here

            print(f"    -> Running colision test ...")

            with rasterio.open(path_basins) as src:
                basins_data = src.read(1)
                transform = src.transform

            # a) Grabs IDS from all basins that touch the bordes of the current matrix (core box + current buffer)
            top_edge = basins_data[0, :]
            bottom_edge = basins_data[-1, :]
            left_edge = basins_data[:, 0]
            right_edge = basins_data[:, -1]

            edge_ids = set(np.unique(np.concatenate([top_edge, bottom_edge, left_edge, right_edge])))
            edge_ids.discard(0) # Removes noData

            # b) Calculates the centroid of each basin present inside the raster file
            unique_ids = np.unique(basins_data)
            unique_ids = unique_ids[unique_ids > 0] # Removes noData (I hope)

            centers = ndimage.center_of_mass(basins_data > 0, basins_data, unique_ids)
```

```
107
108             owned_by_core_ids = set()
109             c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
110
111             # c) Filters which basins have their centroid INSIDE
     core tile
112             for basin_id, (row_center, col_center) in zip(
    unique_ids, centers):
113                 # Converts pixel (line, column) to geo cords (x,
     y in meters)
114                 x_coord, y_coord = transform * (col_center,
    row_center)
115
116                 if (c_minx <= x_coord <= c_maxx) and (c_miny <=
    y_coord <= c_maxy):
117                     owned_by_core_ids.add(basin_id)
118
119             # d) Is there any basin with centroid inside core
    tile that leaks outside of the buffer?
120             leaking_basins = owned_by_core_ids.intersection(
    edge_ids)
121
122             if len(leaking_basins) > 0:
123                 print(f"[FAIL] {len(leaking_basins)} basin(s)
    leaked outside current buffer")
124                 buffer_current += buffer_steps
125
126             else:
127                 print(f"[SUCESS] All interesting basins fit
    within a buffer of {buffer_current}")
128
129                 wbt.depth_in_sink(dem=filename_dem_clean, output
    =filename_depth, zero_background=True)
130
131                 sucess = True
132
133
134
135         except Exception as e:
136             print(f"[ERROR]: {e}")
137             print(f"    -> Ending runtime ...")
138             break
139
140         # File cleanup
```

```
        temp_files = [filename_dem, filename_dem_clean, filename_d8, filename_sinks]
        if not sucess:
            temp_files.extend([filename_basins, filename_depth])
    # Deletes if failed

        for f in temp_files:
            path_f = os.path.join(out_dir, f)
            if os.path.exists(path_f):
                os.remove(path_f)

    if not sucess and buffer_current >= buffer_limit:
        print(f"[WARNING]: Buffer limit reached. Consider revising terrain.")

if True:
    PATH_VRT = '~/Downloads/output.vrt'
    OUT_DIR = 'test_basins_dynamic'
    BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)

    dynamic_cut(
        path_vrt=PATH_VRT,
        out_dir=OUT_DIR,
        core_bounds_deg=BOUNDS_TESTING,
        tile_id="test_01"
    )


```

Here, we chose a small box just to run a quick check. Our output looked something like this:

```
[test_01] Trying to process with a margin of 0.1
    -> Running WhiteboxTools ...
    -> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
    -> Running WhiteboxTools ...
    -> Running colision test ...
[SUCESS] All interesting basins fit within a buffer of 0.2
```



But how can we actually verify that our code is indeed working? Wouldn't it be cool if we could somehow get a visual output of basins and out core box and core box + buffer at the end of runtime?

For that to work, we have to adapt our code a little bit:

```
import os
```

```
import geopandas as gpd
from shapely.geometry import box
from osgeo import gdal
from whitebox import WhiteboxTools
import rasterio
import numpy as np
import scipy.ndimage as ndimage
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.colors import ListedColormap

gdal.UseExceptions()

def plot_tile_diagnostics(
    basins_data,
    transform,
    core_bounds_5800,
    buf_deg_bounds,
    centers,
    unique_ids,
    owned_by_core_ids,
    tile_id,
):
    """
    Takes our results from the dynamic cutting and outputs them as a visual .png file so we can have visual
    proof that all centroids are contained inside the core box
    It is advised to read the dynamic_cut function before reading this one (down below).

    :param basins_data: Our basins raster array
    :param transform: Our transformation matrix (provided by rasterio)
    :param core_bounds_5800: Tuple (minx, miny, maxx, maxy) that defines our core box limits
    :param buf_deg_bounds: Tuple (buf_minx, buf_miny, buf_maxx, buf_maxy) that deines our core box + buffer limits
    :param centers: List with all coordinates (line and column) from each of the basins centroids
    :param unique_ids: Array containing all of the unique basin IDs present in the image (removing 0)
    :param tile_id: Our tile ID

    """
```

```
39
40      # WARNING: DO NOT REMOVE FIG.
41      fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
42
43      # 1. Generating unique colors for each ID
44      # Generates random colors for each ID to try to avoid visual
        problems with neighbor basins
45      max_id = int(basins_data.max())
46      np.random.seed(
47          42
48      )
49      color_array = np.random.rand(max_id + 1, 4)
50      color_array[
51          0
52      ] = [  # Defines background color (ID 0 = noData)
53          0,
54          0,
55          0,
56          0,
57      ]
58      custom_cmap = ListedColormap(color_array)
59
60      # 2. Plots basins raster with custom colors
61      im = ax.imshow(
62          basins_data,
63          cmap=custom_cmap,
64          interpolation='nearest',  # Avoids blurring
65          extent=[
66              transform[2],
67              transform[2] + transform[0] * basins_data.shape[1],
68              transform[5] + transform[4] * basins_data.shape[0],
69              transform[5],
70          ],
71      )
72
73      buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
74      df_buf = gpd.GeoDataFrame(
75          {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy
    )]},
76          crs='EPSG:4326',
77      )
78      b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
    total_bounds
79      c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
```

```
80
81      # Draws Core Box limits
82      core_rect = patches.Rectangle(
83          (c_minx, c_miny),
84          c_maxx - c_minx,
85          c_maxy - c_miny,
86          linewidth=2.5,
87          edgecolor='red',
88          facecolor='none',
89          linestyle='--',
90          label='Core Tile',
91      )
92      ax.add_patch(core_rect)
93
94      # Draws Core Box + Buffer tile
95      buf_rect = patches.Rectangle(
96          (b_minx, b_miny),
97          b_maxx - b_minx,
98          b_maxy - b_miny,
99          linewidth=2,
100         edgecolor='blue',
101         facecolor='none',
102         label='Buffer Tile (Final)',
103     )
104     ax.add_patch(buf_rect)
105
106     # Plots centroids
107     first_core = True
108     for basin_id, (row_center, col_center) in zip(unique_ids,
    centers):
109         x_coord, y_coord = transform * (col_center, row_center)
110
111         if basin_id in owned_by_core_ids:
112             label_text = 'Centroid Core' if first_core else ''
113             ax.plot(
114                 x_coord,
115                 y_coord,
116                 marker='*',
117                 color='black',
118                 markersize=5,
119                 markeredgecolor='white',
120                 markeredgewidth=0.5,
121                 label=label_text,
122             )
```

```
            first_core = False

        else:
            ax.plot(
                x_coord,
                y_coord,
                marker='o',
                color='dimgray',
                markersize=2,
                alpha=0.4,
            )

    # Ajust zoom
    ax.set_xlim(b_minx, b_maxx)
    ax.set_ylim(b_miny, b_maxy)

    # Graphic aesthetics
    ax.set_facecolor('#f4f4f4')
    ax.grid(True, linestyle=':', alpha=0.5, color='gray')
    plt.title(
        f'Basins check - Tile: {tile_id}',
        fontsize=14,
        fontweight='bold',
    )
    plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
    plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
    plt.legend(loc='upper right', frameon=True, facecolor='white
', shadow=True)

    plt.tight_layout()
    plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
    plt.close()
    print(
        f'    -> Image stored as: diagnostic_{tile_id}.png'
    )

def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
    """
    This will cut the DEM according to topography, preserving
basin boundaries (AKA no basins get cut)
    Criteria for cutting: All basins with a centroid that
belongs to core tile A will be grouped together on the same
TIF

```

```
    :param path_vrt: Path to our .vrt file
    :param out_dir: Path for our output
    :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
    :param tile_id: Unique tile ID

    """

    os.makedirs(out_dir, exist_ok=True)
    minx, miny, maxx, maxy = core_bounds_deg

    # Projects Core Tile limist from 4326 (degress) to 5800 (meters)
    df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny, maxx, maxy)]}, crs="EPSG:4326")
    df_core_5800 = df_core.to_crs("EPSG:5880")
    core_bounds_5800 = df_core_5800.total_bounds # (minx, miny, maxx, maxy) em metros

    # Dynamic buffer initial setup
    buffer_current = 0.1
    buffer_steps = 0.1
    buffer_limit = 4
    sucess = False

    wbt = WhiteboxTools()
    wbt.set_working_dir(os.path.abspath(out_dir))
    wbt.verbose = False

    while not sucess and buffer_current <= buffer_limit:
        print(f"[{tile_id}] Trying to process with a margin of {buffer_current}")

        # Define limits
        buf_minx = minx - buffer_current
        buf_miny = miny - buffer_current
        buf_maxx = maxx + buffer_current
        buf_maxy = maxy + buffer_current

        # Names of temporary files
        filename_dem = f"temp_{tile_id}_dem.tif"
        filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
        filename_d8 = f"temp_{tile_id}_d8.tif"
        filename_sinks = f"temp_{tile_id}_sinks.tif"
        filename_basins = f"basins_final_{tile_id}.tif"
```

```
203         filename_depth = f"depth_final_{tile_id}.tif"
204 
205         path_dem = os.path.join(out_dir, filename_dem)
206         path_basins = os.path.join(out_dir, filename_basins)
207 
208         try:
209             # Cuts OG VRT
210             gdal.Warp(
211                 destNameOrDestDS=path_dem,
212                 srcDSOrSrcDSTab=path_vrt,
213                 format='GTiff',
214                 outputBounds=(buf_minx, buf_miny, buf_maxx,
    buf_maxy),
215                 outputBoundsSRS='EPSG:4326',
216                 dstSRS='EPSG:5880',
217                 xRes=30, yRes=30,
218                 creationOptions=['COMPRESS=DEFLATE', 'TILED=YES'
    ]
219             )
220 
221             # WhiteboxTools pipeline here
222             print(f"    -> Running WhiteboxTools ...")
223             wbt.breach_single_cell_pits(dem=filename_dem, output
    =filename_dem_clean)
224             wbt.d8_pointer(dem=filename_dem_clean, output=
    filename_d8)
225             wbt.sink(i=filename_dem_clean, output=filename_sinks
    , zero_background=True)
226             wbt.watershed(d8_pntr=filename_d8, pour_pts=
    filename_sinks, output=filename_basins)
227 
228             # Colision Test pipeline here
229 
230             print(f"    -> Running colision test ...")
231 
232             with rasterio.open(path_basins) as src:
233                 basins_data = src.read(1)
234                 transform = src.transform
235 
236             # a) Grabs IDS from all basins that touch the bordes
     of the current matrix (core box + current buffer)
237             top_edge = basins_data[0, :]
238             bottom_edge = basins_data[-1, :]
239             left_edge = basins_data[:, 0]
```

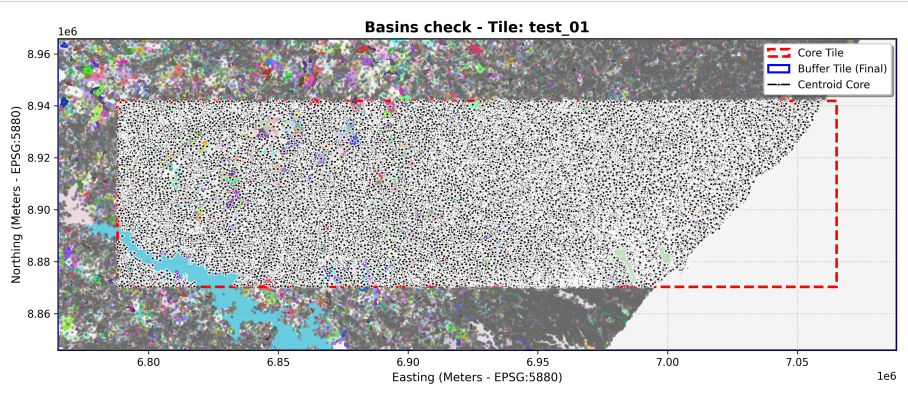
```
            right_edge = basins_data[:, -1]

            edge_ids = set(np.unique(np.concatenate([top_edge, bottom_edge, left_edge, right_edge])))
            edge_ids.discard(0) # Removes noData

            # b) Calculates the centroid of each basin present inside the raster file
            unique_ids = np.unique(basins_data)
            unique_ids = unique_ids[unique_ids > 0] # Removes noData (I hope)

            centers = ndimage.center_of_mass(basins_data > 0, basins_data, unique_ids)

            owned_by_core_ids = set()
            c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800

            # c) Filters which basins have their centroid INSIDE core tile
            for basin_id, (row_center, col_center) in zip(unique_ids, centers):
                # Converts pixel (line, column) to geo cords (x, y in meters)
                x_coord, y_coord = transform * (col_center, row_center)

                if (c_minx <= x_coord <= c_maxx) and (c_miny <= y_coord <= c_maxy):
                    owned_by_core_ids.add(basin_id)

            # d) Is there any basin with centroid inside core tile that leaks outside of the buffer?
            leaking_basins = owned_by_core_ids.intersection(edge_ids)

            if len(leaking_basins) > 0:
                print(f"[FAIL] {len(leaking_basins)} basin(s) leaked outside current buffer")
                buffer_current += buffer_steps

            else:
                print(f"[SUCESS] All interesting basins fit within a buffer of {buffer_current}")
```

```
271
272                 wbt.depth_in_sink(dem=filename_dem_clean, output
    =filename_depth, zero_background=True)
273
274                 # testing quick visuals
275                 print(f"    -> Generating image ...")
276                 buf_bounds = (buf_minx, buf_miny, buf_maxx,
    buf_maxy)
277                 plot_tile_diagnostics(
278                     basins_data,
279                     transform,
280                     core_bounds_5800,
281                     buf_bounds,
282                     centers,
283                     unique_ids,
284                     owned_by_core_ids,
285                     tile_id,
286                 )
287                 sucess = True
288
289
290
291         except Exception as e:
292             print(f"[ERROR]: {e}")
293             print(f"    -> Ending runtime ...")
294             break
295
296         #
297         temp_files = [filename_dem, filename_dem_clean,
    filename_d8, filename_sinks]
298         if not sucess:
299             temp_files.extend([filename_basins, filename_depth])
     # Deletes if failed
300
301         for f in temp_files:
302             path_f = os.path.join(out_dir, f)
303             if os.path.exists(path_f):
304                 os.remove(path_f)
305
306     if not sucess and buffer_current >= buffer_limit:
307         print(f"[WARNING]: Buffer limit reached. Consider
    revising terrain.")
308
309 if True:
```

```
    PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
    OUT_DIR = 'test_basins_dynamic'
    BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)

    dynamic_cut(
        path_vrt=PATH_VRT,
        out_dir=OUT_DIR,
        core_bounds_deg=BOUNDS_TESTING,
        tile_id="test_01"
    )


```

What this code does is basically, at the end of the loop, if we get a sucess, we call a funtion to generate a visual graph of our results, and output it as a .png file.

Output for this code looked this:




```
[test_01] Trying to process with a margin of 0.1
    -> Running WhiteboxTools ...
    -> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
    -> Running WhiteboxTools ...
    -> Running colision test ...
[SUCESS] All interesting basins fit within a buffer of 0.2
    -> Generating image ...
    -> Image stored as: diagnostic_test_01.png
```

As we can see, all the basins with a centroid belonging to the original core box are indeed inside the bigger core box + buffer area. Therefore we can be sure that those ones will not suffer from boundary limitations and exclude the others.

To remove those other basins, we adapt the code some more:

```
import os
import geopandas as gpd
from shapely.geometry import box
from osgeo import gdal
from whitebox import WhiteboxTools
import rasterio
import numpy as np
import scipy.ndimage as ndimage
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.colors import ListedColormap

gdal.UseExceptions()

def plot_tile_diagnostics(
    basins_data,
    transform,
    core_bounds_5800,
    buf_deg_bounds,
    centers,
    unique_ids,
    owned_by_core_ids,
    tile_id,
):
    """
    Takes our results from the dynamic cutting and outputs them as a visual .png file so we can have visual
    proof that all centroids are contained inside the core box
    It is advised to read the dynamic_cut function before reading this one (down below).

    :param basins_data: Our basins raster array
    :param transform: Our transformation matrix (provided by rasterio)
    :param core_bounds_5800: Tuple (minx, miny, maxx, maxy) that defines our core box limits
    :param buf_deg_bounds: Tuple (buf_minx, buf_miny, buf_maxx, buf_maxy) that deines our core box + buffer limits
    :param centers: List with all coordinates (line and column)
```

```
    from each of the basins centroids
35      :param unique_ids: Array containing all of the unique basin
    IDs present in the image (removing 0)
36      :param tile_id: Our tile ID
37
38      """
39
40      # WARNING: DO NOT REMOVE FIG.
41      fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
42
43      # 1. Generating unique colors for each ID
44      # Generates random colors for each ID to try to avoid visual
        problems with neighbor basins
45      max_id = int(basins_data.max())
46      np.random.seed(
47          42
48      )
49      color_array = np.random.rand(max_id + 1, 4)
50      color_array[
51          0
52      ] = [  # Defines background color (ID 0 = noData)
53          0,
54          0,
55          0,
56          0,
57      ]
58      custom_cmap = ListedColormap(color_array)
59
60      # 2. Plots basins raster with custom colors
61      im = ax.imshow(
62          basins_data,
63          cmap=custom_cmap,
64          interpolation='nearest',  # Avoids blurring
65          extent=[
66              transform[2],
67              transform[2] + transform[0] * basins_data.shape[1],
68              transform[5] + transform[4] * basins_data.shape[0],
69              transform[5],
70          ],
71      )
72
73      buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
74      df_buf = gpd.GeoDataFrame(
75          {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy
```

```
    )]},
76          crs='EPSG:4326',
77      )
78      b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
    total_bounds
79      c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
80
81      # Draws Core Box limits
82      core_rect = patches.Rectangle(
83          (c_minx, c_miny),
84          c_maxx - c_minx,
85          c_maxy - c_miny,
86          linewidth=2.5,
87          edgecolor='red',
88          facecolor='none',
89          linestyle='--',
90          label='Core Tile',
91      )
92      ax.add_patch(core_rect)
93
94      # Draws Core Box + Buffer tile
95      buf_rect = patches.Rectangle(
96          (b_minx, b_miny),
97          b_maxx - b_minx,
98          b_maxy - b_miny,
99          linewidth=2,
100         edgecolor='blue',
101         facecolor='none',
102         label='Buffer Tile (Final)',
103     )
104     ax.add_patch(buf_rect)
105
106     # Plots centroids
107     first_core = True
108     for basin_id, (row_center, col_center) in zip(unique_ids,
    centers):
109         x_coord, y_coord = transform * (col_center, row_center)
110
111         if basin_id in owned_by_core_ids:
112             label_text = 'Centroid Core' if first_core else ''
113             ax.plot(
114                 x_coord,
115                 y_coord,
116                 marker='',
```

```
                color='black',
                markersize=5,
                markeredgecolor='white',
                markeredgewidth=0.5,
                label=label_text,
            )
            first_core = False

        else:
            ax.plot(
                x_coord,
                y_coord,
                marker='',
                color='dimgray',
                markersize=2,
                alpha=0.4,
            )

    # Ajust zoom
    ax.set_xlim(b_minx, b_maxx)
    ax.set_ylim(b_miny, b_maxy)

    # Graphic aesthetics
    ax.set_facecolor('#f4f4f4')
    ax.grid(True, linestyle=':', alpha=0.5, color='gray')
    plt.title(
        f'Basins check - Tile: {tile_id}',
        fontsize=14,
        fontweight='bold',
    )
    plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
    plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
    plt.legend(loc='upper right', frameon=True, facecolor='white
   ', shadow=True)

    plt.tight_layout()
    plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
    plt.close()
    print(
        f'    -> Image stored as: diagnostic_{tile_id}.png'
    )

def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
    """
```

```
    This will cut the DEM according to topography, preserving basin boundaries (AKA no basins get cut)
    Criteria for cutting: All basins with a centroid that belongs to core tile A will be grouped together on the same TIF

    :param path_vrt: Path to our .vrt file
    :param out_dir: Path for our output
    :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
    :param tile_id: Unique tile ID

    """

    os.makedirs(out_dir, exist_ok=True)
    minx, miny, maxx, maxy = core_bounds_deg

    # Projects Core Tile limist from 4326 (degress) to 5800 (meters)
    df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny, maxx, maxy)]}, crs="EPSG:4326")
    df_core_5800 = df_core.to_crs("EPSG:5880")
    core_bounds_5800 = df_core_5800.total_bounds # (minx, miny, maxx, maxy) em metros

    # Dynamic buffer initial setup
    buffer_current = 0.1
    buffer_steps = 0.1
    buffer_limit = 4
    sucess = False

    wbt = WhiteboxTools()
    wbt.set_working_dir(os.path.abspath(out_dir))
    wbt.verbose = False

    while not sucess and buffer_current <= buffer_limit:
        print(f"[{tile_id}] Trying to process with a margin of {buffer_current}")

        # Define limits
        buf_minx = minx - buffer_current
        buf_miny = miny - buffer_current
        buf_maxx = maxx + buffer_current
        buf_maxy = maxy + buffer_current

```

```
197         # Names of temporary files
198         filename_dem = f"temp_{tile_id}_dem.tif"
199         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
200         filename_d8 = f"temp_{tile_id}_d8.tif"
201         filename_sinks = f"temp_{tile_id}_sinks.tif"
202         filename_basins = f"basins_final_{tile_id}.tif"
203         filename_depth = f"depth_final_{tile_id}.tif"
204 
205         path_dem = os.path.join(out_dir, filename_dem)
206         path_basins = os.path.join(out_dir, filename_basins)
207 
208         try:
209             # Cuts OG VRT
210             gdal.Warp(
211                 destNameOrDestDS=path_dem,
212                 srcDSOrSrcDSTab=path_vrt,
213                 format='GTiff',
214                 outputBounds=(buf_minx, buf_miny, buf_maxx,
    buf_maxy),
215                 outputBoundsSRS='EPSG:4326',
216                 dstSRS='EPSG:5880',
217                 xRes=30, yRes=30,
218                 creationOptions=['COMPRESS=DEFLATE', 'TILED=YES'
    ]
219             )
220 
221             # WhiteboxTools pipeline here
222             print(f"    -> Running WhiteboxTools ...")
223             wbt.breach_single_cell_pits(dem=filename_dem, output
    =filename_dem_clean)
224             wbt.d8_pointer(dem=filename_dem_clean, output=
    filename_d8)
225             wbt.sink(i=filename_dem_clean, output=filename_sinks
    , zero_background=True)
226             wbt.watershed(d8_pntr=filename_d8, pour_pts=
    filename_sinks, output=filename_basins)
227 
228             # Colision Test pipeline here
229 
230             print(f"    -> Running colision test ...")
231 
232             with rasterio.open(path_basins) as src:
233                 basins_data = src.read(1)
234                 transform = src.transform
```

```
235
236             # a) Grabs IDS from all basins that touch the bordes
    of the current matrix (core box + current buffer)
237             top_edge = basins_data[0, :]
238             bottom_edge = basins_data[-1, :]
239             left_edge = basins_data[:, 0]
240             right_edge = basins_data[:, -1]
241
242             edge_ids = set(np.unique(np.concatenate([top_edge,
    bottom_edge, left_edge, right_edge])))
243             edge_ids.discard(0) # Removes noData
244
245             # b) Calculates the centroid of each basin present
    inside the raster file
246             unique_ids = np.unique(basins_data)
247             unique_ids = unique_ids[unique_ids > 0] # Removes
    noData (I hope)
248
249             centers = ndimage.center_of_mass(basins_data > 0,
    basins_data, unique_ids)
250
251             owned_by_core_ids = set()
252             c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
253
254             # c) Filters which basins have their centroid INSIDE
     core tile
255             for basin_id, (row_center, col_center) in zip(
    unique_ids, centers):
256                 # Converts pixel (line, column) to geo cords (x,
     y in meters)
257                 x_coord, y_coord = transform * (col_center,
    row_center)
258
259                 if (c_minx <= x_coord <= c_maxx) and (c_miny <=
    y_coord <= c_maxy):
260                     owned_by_core_ids.add(basin_id)
261
262             # d) Is there any basin with centroid inside core
    tile that leaks outside of the buffer?
263             leaking_basins = owned_by_core_ids.intersection(
    edge_ids)
264
265             if len(leaking_basins) > 0:
266                 print(f"[FAIL] {len(leaking_basins)} basin(s)
```

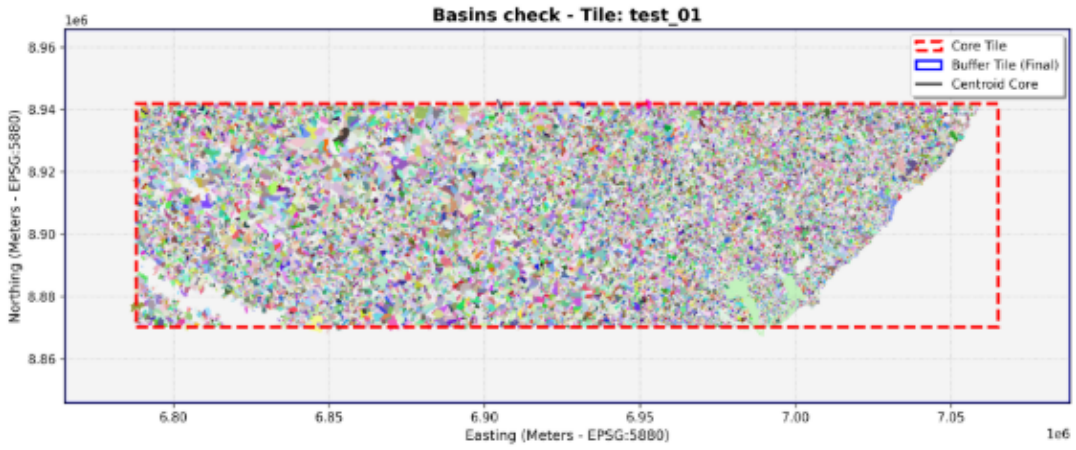
```
leaked outside current buffer")
267                 buffer_current += buffer_steps
268 
269             else:
270                 print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
271 
272                 wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
273 
274                 # Removing excess basins
275                 print(f"    -> Applying mask to keep only basins
 of interest ...")
276 
277                 # a) Transforms the valid IDs set into a numpy
array
278                 core_ids_array = np.array(list(owned_by_core_ids
))
279 
280                 # b) Creates a boolean mask (True if the pixel
belongs to a basin of interest)
281                 mask = np.isin(basins_data, core_ids_array)
282 
283                 # c) Checks which noData value is used by DEM
and applies mask over basin_data
284                 with rasterio.open(path_basins, 'r+') as dst:
285                     lines, columns = basins_data.shape
286                     area_m_square = (lines * 30) * (columns *
30)
287                     print(f"    -> Total area processed: {
area_m_square / 1000000:.2f} km2")
288                     nodata_val = dst.nodata if dst.nodata is not
 None else 0
289 
290                     filtered_basins = np.where(mask, basins_data
, nodata_val)
291                     dst.write(filtered_basins, 1)
292 
293                 # d) Open, filers and overwrites depth TIF
294                 path_depth = os.path.join(out_dir,
filename_depth)
295                 with rasterio.open(path_depth, 'r+') as dst:
296                     depth_data = dst.read(1)
297                     nodata_val_depth = dst.nodata if dst.nodata
```

```
    is not None else 0
298
299                     filtered_depth = np.where(mask, depth_data,
    nodata_val_depth)
300                     dst.write(filtered_depth, 1)
301
302                 # testing quick visuals
303                 print(f"    -> Generating image ...")
304                 buf_bounds = (buf_minx, buf_miny, buf_maxx,
    buf_maxy)
305                 plot_tile_diagnostics(
306                     filtered_basins,
307                     transform,
308                     core_bounds_5800,
309                     buf_bounds,
310                     centers,
311                     unique_ids,
312                     owned_by_core_ids,
313                     tile_id,
314                 )
315                 sucess = True
316
317
318
319         except Exception as e:
320             print(f"[ERROR]: {e}")
321             print(f"    -> Ending runtime ...")
322             break
323
324         #
325         temp_files = [filename_dem, filename_dem_clean,
    filename_d8, filename_sinks]
326         if not sucess:
327             temp_files.extend([filename_basins, filename_depth])
     # Deletes if failed
328
329         for f in temp_files:
330             path_f = os.path.join(out_dir, f)
331             if os.path.exists(path_f):
332                 os.remove(path_f)
333
334     if not sucess and buffer_current >= buffer_limit:
335         print(f"[WARNING]: Buffer limit reached. Consider
    revising terrain.")
```

```
336
337 if True:
338     PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
339     OUT_DIR = 'test_basins_dynamic'
340     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
341
342     dynamic_cut(
343         path_vrt=PATH_VRT,
344         out_dir=OUT_DIR,
345         core_bounds_deg=BOUNDS_TESTING,
346         tile_id="test_01"
347     )
348
349
```

This will give us our final output for this subsection:

```
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ python3 terrain_cut_centroid.py
[test_01] Trying to process with a margin of 0.1
    -> Running WhiteboxTools ...
    -> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
    -> Running WhiteboxTools ...
    -> Running colision test ...
[SUCESS] All interesting basins fit within a buffer of 0.2
    -> Applying mask to keep only basins of interest ...
    -> Total area processed: 38796.10 km2
    -> Generating image ...
    -> Image stored as: diagnostic_test_01.png
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$
```



As we can see, we no longer have those extra basins on our final file!

2.4 Analysing basins and handling infiltration

So now we have it: we can actually find basins! But how do we analyse each one of them?

First, we have to look at our last program's outputs (in this case, the tif files):

- basins final → tells us where basins are.
- depth final → tells us the max depth of each of the basins pieces until it overflows.

Here, the secret is to use NumPy masks. We tell Python to isolate only the pixels belonging to a specific ID, and we run calculations with only those pixels.

But before we get anything started, we have to address the elephant in the room: rain water infiltration. As you may already guess, soil is permeable → this means that water will only start overflowing when terrain gets too saturated. Not only that, but different terrains will behave in different ways when rainwater falls, where some will soak a lot of volume but others will basically be impermeable. So this raises a question: How can we deal with this problem?

This rushes us to present another very cool thing in hydrology: the Curve Number (CN) method. It was developed by the US Department of Agriculture (USDA) Natural Resources Conservation Service (NRCS) (back when it was formerly known as the Soil Conservation Service (SCS)). The SCS CN method estimates precipitation excess as a function of the cumulative precipitation depth, soil cover, land use, and antecedent soil mosture as:

$$P_e = \begin{cases} 0, & \text{for } P < I_a \\ \dfrac{(P - I_a)^2}{P - I_a + S}, & \text{otherwise} \end{cases} \tag{1}$$

where P_e = accumulated precipitation excess at time t; P = accumulated rainfall depth at time t; I_a = the initial abstraction (initial loss); and S = potential maximum retention, a meaasure of the ability of a watershed to abstract and retain storm precipitation. Until the accumulated rainfall exceeds the initial abstraction, the precipitation excess, hence the runoff, will be zero. From analysis of results from many small experimental watersheds, the SCS developed an empirical relation of I_a and S:

$$I_a = 0.2 * S \tag{2}$$

Therefore, the cumulative excess as time t is:

$$P_e = \frac{(P - 0.2 * S)^2}{(P + 0.8 * S)} \tag{3}$$

Incremental excess for a time interval is computed as the difference between the accumulated excess at the end of and beginning of the period. The maximum retention, S, and watershed characteristics are related through an intermediate paramenter, the curve number (commonly abbreviated CN) as:

$$S = \begin{cases} \dfrac{1000 - 10 * CN}{CN}, & \text{foot / pound system} \\ \dfrac{25400 - 254 * CN}{CN}, & \text{The obviously better alternative, also known as SI} \end{cases} \tag{4}$$

Jokes aside, CN values range from 100 (for water bodies) to approximately 30 for permeable soils with high infiltration rates.

The curve number that is entered should be a "composite" curve number that represents all of the different soil groups and land use combinations in the subbasin. Typically, curve numbers are derive from soil maps. I_a defines the ammount of precipitation that must fall before excess precipitation results. Here, we will use (2) (as provided by the curve number provider that we will mention shortly).

For a watershed that consists of several soil types and land uses, a composite CN is calculated as:

$$CN_{composite} = \frac{\sum A_i CN_i}{\sum A_i} \tag{5}$$